

Session Notes: Visual Design, Design Sprints and LLM Fundamentals — Mastering Product Management (MPM) Program

Session Details

Session Title: Visual Design, Design Sprints and LLM Fundamentals

Instructor(s): Shravan

Topics Covered:

- Presentation and discovery principles: hooking the listener, sub-segmentation, inference vs insight
- Three lenses on a business: business metrics, product metrics, marketing metrics
- What makes a metric: the A-by-B rule, CAC, and pricing models
- The PRD: balancing thoroughness and succinctness, scope creep
- Principles of visual design: proximity, similarity, responsive design, copy, accessibility
- Principles of composition: unity, contrast, balance, use of space
- The fidelity-interactivity workflow: sketches, wireframes, mockups, live products
- The design sprint: five-day structure, roles, and decision methods
- The product-design-engineering workflow: concept note, BRD draft, engineering handshake
- AI-era prototyping and the shift from chat to work
- From analytics to intelligence: descriptive, predictive and prescriptive analytics
- Machine learning, neural networks and backpropagation
- Large language models: scale, scaling laws, training stages, mixture of experts
- Emergent behaviour and generative AI
- Tokens as the currency of LLMs; prompting and token efficiency
- Hallucination, interpretability, and how reasoning works
- Transformers and the attention architecture
- Case study briefing: India's elder care ecosystem

Executive Summary

This session bridges two worlds a modern product manager must operate in: the craft of design and the machinery of large language models. It completes the design curriculum begun in the previous session and then opens the first hardcore theory block on how LLMs actually work, so that later application-focused sessions on Claude Code rest on genuine understanding rather than recipes.

The design half moves from principles to process. Visual design principles — proximity, similarity, responsiveness, and the quality of copy — explain why an Amazon product page places Add to Cart and Buy Now together, why identical-looking buttons must behave identically, and why a single word like "enroll" versus "buy" changes how users perceive a product. Composition principles (unity, contrast, balance, use of space) provide a checklist for judging whether a page feels coherent. The process layer covers the fidelity-interactivity matrix (why changes get exponentially costlier as designs approach

production), the five-day design sprint with its decider, facilitator and voting mechanics, and the correct product-design-engineering workflow: a one-page concept note, a commentable BRD draft, parallel design work, and an explicit engineering handshake before any sprint begins. A live five-minute sketching exercise drives home how hard it is to translate an idea into a design, and why designers deserve realistic timelines.

The LLM half builds from first principles: Alan Turing's 1950 question "Can machines think?", the distinction between knowledge and intelligence, the three types of analytics, and the neural network with backpropagation at its core. The AI winter ended when data became cheap (Jio) and parallel compute became available (NVIDIA's CUDA), enabling models to scale from GPT-era millions of parameters to today's trillions. The session explains pre-training, fine-tuning and post-training; scaling laws versus the cost-efficiency philosophy of Chinese labs; emergent behaviour; tokens as vectorised representations and why common vocabulary makes prompting cheaper; hallucination; interpretability research; and the define-do-verify loop that underlies reasoning levels. It closes with the transformer architecture from the 2017 paper "Attention Is All You Need" and a new case study on India's elder care market, due 1 August.

Detailed Topic Notes

1. Presenting Discovery Work: The Hook, Sub-Segmentation and Insight

What is it? A set of principles for presenting discovery research so that the depth of insight — not theatrics — carries the presentation, drawn from a full round of team presentations on the Indian convenience economy.

Why does it matter? The goal of any presentation is not to convince the listener within two minutes; it is to hook them into listening further. This is the true meaning of an elevator pitch: within two or three minutes, you earn the right to the rest of the conversation. Depth is the signal a presentation must carry — the sense that "we found something genuinely interesting." Everything else, including humour and production value, is decoration around that core.

How does it work?

- **Lead with the insight.** The sharpest thing you discovered should be unmistakable. Fun and storytelling are welcome, but only wrapped around a real insight.
- **Sub-segment the problem.** A large, high-level market number is a superset; it rarely maps to the specific problem being solved. Break a wide problem into sub-buckets and size each one, because the size of each bucket determines whether the opportunity is real. Claiming "there is a market of 1 billion dollars" and letting it trickle down to a narrow solution confuses the superset with the addressable segment.
- **Inference is not insight.** "Everyone needs medicines" is an inference; it does not follow that a medicine startup is a good idea. An insight ties observation to a specific, evidenced opportunity. Wherever possible, replace subjectivity with data — data is what brings objectivity. If data genuinely does not exist, that is a finding in itself, but in most cases people simply have not done deep enough research.
- **Every proposition needs foolproof logic.** A lending idea, for example, must answer the recovery question: credit is only extended where a recovery mechanism exists — this is exactly why a street hawker cannot get a loan. Without underwriting logic, even a 10 percent default rate becomes thousands of crores at aggregate scale. One startup in the vehicle-financing space solved this by selling vehicles to the aggregator platform itself rather than lending to individual workers, offloading the default risk entirely.

- **Friction can be a conscious design choice.** Some systems are deliberately hard to use — withdrawing from a provident fund is difficult by design. When research uncovers a gap, ask whether the gap is an oversight or a conscious choice by the incumbent, because that determines whether the incumbent will welcome or resist a fix.

Examples and Case Studies: The discovery arc of the program itself: the first phase focuses purely on discovery, but by the final phase teams must carry a product all the way to real users — discovery, execution, flow, and a genuine product, not merely a prototype. A related professional application: before any product interview or investor pitch, research the individual — their LinkedIn history, their articles, their past investments. A person's background is an 80 percent signal of what they will ask. This is why founders on Shark Tank reference an investor's previous investments: it demonstrates judicious research. Speaking in front of a hundred people is genuinely hard — everyone shakes the first time, the way everyone falls the first time they ride a bicycle — and the merit of a conversation is never in pedigree; it is in the essence of the conversation itself.

***Key Insight:** A presentation succeeds when depth of insight does the persuading — hook the listener in the first two minutes, sub-segment every big number, and never confuse an inference with an insight.*

2. Three Lenses on a Business: Business, Product and Marketing Metrics

What is it? A framework for viewing any product system through three complementary metric families: business metrics, product metrics, and marketing metrics.

Why does it matter? A system — like a house being built — can be examined through multiple dimensions: budget, structure, usage. A good operator looks at a business through all its lenses simultaneously, because each lens alone can mislead. This is also the backbone of the BRD: a business requirements document captures business metrics, usually usage metrics, and often go-to-market (marketing) metrics.

How does it work?

Lens	What it measures	Representative metrics
Business metrics	Whether the business makes money	Revenue, cost, COGS (cost of goods sold), service cost, margins (contribution margin CM1, CM2, CM3), EBITDA
Product metrics	Whether the product is used	Retention, churn, activation, acquisition, funnels, engineering metrics
Marketing metrics	Whether growth is efficient	CAC, LTV (lifetime value), CPI (cost per impression), cost per lead, cost per sale

Supporting concepts:

- **EBITDA and amortization.** The A in EBITDA is amortization — depreciation of intangible assets. A balance sheet has four elements: income, expenses, assets, and liabilities. Assets are not costs: a Rs 5 lakh laptop or a Rs 10 lakh car is an asset whose value depreciates over time (the car may be worth Rs 8 lakh after a year), and it can be sold later. This is why balance sheets list assets separately.
- **Usage usually reflects business — but not always.** Government websites are the exception: they earn from mandatory services regardless of usage quality, because users have no alternative.

In most markets, without usage there is no revenue, which is why free products only have a life while they remain free — the moment they charge, most users do not pay.

- **Role-dependent emphasis.** A growth PM lives mostly in marketing and product metrics; a technical or platform PM lives in engineering metrics (a subset of product metrics). But a full systemic operator needs a sense of all three: marketing tells you how people arrive, product tells you whether they use it, and business tells you whether it is profitable to run.

Examples and Case Studies: Every CEO keynote demonstrates the three lenses in one breath: "We acquired this many users, our usage grew this much, and our revenue grew this much." The three are never independent — a product with good product metrics but bad business metrics is not a scalable system.

***Key Insight:** Business, product and marketing metrics are three views of the same system; a product only truly works when all three lenses agree, and a complete PM can read all of them.*

3. What Makes a Metric: The A-by-B Rule, CAC and Pricing Models

What is it? The definition of a metric, the discipline of expressing metrics as ratios, and a worked understanding of customer acquisition cost and subscription pricing.

Why does it matter? The word metric derives from the Greek "metron" — to measure. A metric is only valuable when it is mathematical: if you cannot quantify an improvement, you cannot claim it. Absolute numbers hide bias; ratios remove it.

How does it work?

- **Express metrics as A by B.** Revenue alone is a bad metric — revenue can grow 500x while the business loses money. "10 million users" is meaningless if every user bleeds money. Ratios such as revenue per user or retention per month carry real information.
- **Retention needs a denominator.** There is no such thing as bare "retention". It is per day, per month, per quarter, or per year — and yearly retention is a vastly different thing from daily retention. A resume line reading "improved retention by 20%" with no time denominator signals that the writer does not understand the metric.
- **CAC (Customer Acquisition Cost).** A customer is a paying person; someone who downloads a PDF is a lead, not a customer. Worked example: suppose 900 applicants are each acquired at Rs 1,000 of marketing spend (Rs 9 lakh total), and 90 become paying students. The CAC is roughly Rs 10,000 per customer — the cost of ten leads produces one customer. Lead quality varies enormously: someone who has followed an educator's content for years is a far higher-quality lead than someone who grabbed a free download. CAC also covers freemium models — the cost of acquiring users who churn during a free period is still part of CAC.
- **Monthly vs yearly pricing.** New brands without trust use monthly pricing: a user will commit Rs 500 a month but never Rs 20,000 upfront to an unknown, so the brand hopes to recover its CAC over 20-25 months — at the risk that the user unsubscribes after three months and the brand loses money. Established brands can charge yearly and upfront, because accumulated credibility earns the larger commitment. Which model dominates depends on the product's lifecycle and its value in the ecosystem — some strong brands report yearly as their most-subscribed plan; others report monthly.

Examples and Case Studies: A cohort-based education program received 700-800 applications and admitted around 90, consciously capping the batch rather than admitting 10-20 more paying students, because serving more students well would have required more mentors and more attention than the

system could deliver. That is a business decision made through the service-quality lens, not the revenue lens.

Key Insight: *A metric without a denominator is an opinion — always express metrics as A by B, and remember that a customer is someone who pays, not someone who clicks.*

4. The PRD: Thorough Yet Succinct

What is it? The core quality bar for a product requirements document: it must be simultaneously comprehensive and readable — two opposing forces held in balance.

Why does it matter? The test of a good PRD is simple: once developers start working from it, they should have the fewest possible questions. If a PRD has to be explained again and again, the writing failed. Every re-explanation is "lost in translation" — a loss of energy from the system, exactly like any transmission loss.

How does it work?

- **Two failure modes.** A PRD that is too long — 100-page essays — goes unread, and the implementation drifts. A PRD that is too thin misses parallel edge cases. Both produce lost-in-translation failures.
- **The orbit analogy.** Thoroughness and succinctness are opposing forces that must cancel each other out, the way centrifugal force and gravity cancel to keep a body in orbit. The document should be thorough enough to give the total essence, succinct enough to be readable. As a rule of thumb, no PRD should exceed 18-25 pages.
- **Long documents cause scope creep.** Scope creep happens because of long documents that nobody fully implements. The identical philosophy applies to LLM prompts: an LLM reads the prompt exactly as a developer reads a PRD. The longer the prompt, the more non-directional words creep in — like two separate implementations of the same feature in one codebase, leaving the model unsure which to follow.
- **First principles transfer.** The same principles recur across engineering, product management, design and prompting. The best practitioners in the world all say the same things: writing matters, distribution matters, clarity matters. Learn the first principle once and it applies everywhere — including to LLMs.

Examples and Case Studies: Writing a PRD improves exactly like a neural network trains: the first version is weak, feedback comes back ("if the objective section is two pages, nobody will read it"), the next version has a one-paragraph objective, and iteration by iteration the internal model of a good PRD strengthens. This parallel is developed further in the neural network topic below.

Key Insight: *A PRD is measured by the questions it prevents — make it thorough enough to carry the full essence and succinct enough to actually be read, because the same document discipline governs both engineers and LLMs.*

5. Principles of Visual Design: Proximity, Similarity, Responsiveness and Copy

What is it? The working principles of visual design — the topmost layer of the design iceberg above conceptual, interaction and information design — that determine how users read importance and function from a screen.

Why does it matter? General design principles (usability, efficiency, perceived effort, simplicity with delight) assess overall design. Visual design principles answer a narrower question: if something must be shown visually, how should it be shown so users instantly understand what matters and what does what?

How does it work?

- **Proximity principle.** Objects that share visual characteristics and sit close together are read as similarly important. On an Amazon product page, Add to Cart and Buy Now differ slightly in colour but match in size and sit adjacent — the two most important actions on the page. This is also why Amazon and Flipkart measure add-to-cart: cart adders sit close to the buying funnel, and one well-timed discount notification (an iPhone drops Rs 10,000 in a sale) converts them. By contrast, the wishlist action is deliberately de-emphasised — most users cannot even locate it on the page — because a wishlist signals "not in my purchasing power today", a much weaker conversion signal than add-to-cart.
- **Similarity principle.** Elements that look the same are perceived as — and must be — functionally the same. Size-selector chips all behave identically; the Add to Cart button at the top of a page and the one at the bottom do the same thing; two similar-looking "join waitlist" buttons on a website trigger the same action even if the copy differs slightly. Violating this breaks user trust in the interface.
- **Responsive design.** A product must be visible, valuable and usable on all screen types. A practical check: right-click, Inspect, and switch device dimensions (for example iPhone 14 Pro Max) to preview how a page wraps. A live illustration of failure: a case-study submission feature whose submit button sat off-viewport on smaller screens, so users never found it. Designers will ask "mobile first or desktop first?" — the honest answer for most products is both first. Some companies deliberately block one interface ("better on the app" pop-ups) to concentrate usage, which is itself a responsive-design decision.
- **Marketing and copy.** Copy means the text on the product, and it can do wonders when used judiciously. Small words change how people approach things: an experience product says "enroll" even though the user is fundamentally buying, because enrolling reframes the relationship; Zepto never says "enroll now" for a Rs 300 grocery order because nothing is being enrolled into. Words must not be used interchangeably — every word has its own meaning and connotation (the emotional register in which it is used). Compare two push notifications: "Buy a gift for your mother" versus "Give your mother the gift of surprise" — the emotional copy wins clicks. Naming is copy too: a discovery feature deliberately named "signal", or a company named Rethink Systems precisely because its thesis is rethinking systems. Choosing a name takes real time because the name carries the essence. Good design teams embed copywriters, because copy is the soul of design — a picture is worth a thousand words, but the picture and the words must land together, as in a delivery advertisement whose image of a standing rider pairs with the line "stand and deliver".
- **Accessibility.** Accessibility falls fully under UX. The goal of UX is the easiest, simplest path for a person to find the product's value; if a user cannot work out what to do within minutes, the product has failed regardless of its underlying quality. Audience calibration matters: OpenRouter's interface — reasoning levels and model routing — reads as impenetrable to a lay user but is perfectly clear to its actual audience of deep-tech developers using LLMs.

Examples and Case Studies: The three qualities of a great product frame all of this: significant value-based differentiation, simplicity of usage, and joy of usage. Presentations in the session illustrated joy of usage directly — the same content delivered dryly would have bored the room, and delivered with craft it delighted. And an important caveat: you optimise for 80-85 percent of the audience, never 100 percent; some fraction will always be unmoved.

Key Insight: Users read importance from proximity, function from similarity, and meaning from copy — visual design is the discipline of making those three signals deliberate instead of accidental.

6. Principles of Composition: Unity, Contrast, Balance and Use of Space

What is it? Four all-encompassing principles for judging whether a visual design holds together as a whole: unity, contrast, balance, and use of space.

Why does it matter? Individual elements can each be well designed while the page still feels wrong. Composition principles diagnose page-level coherence — the difference between a page and a bunch of disparate elements.

How does it work?

- **Unity.** Does the page read as one page? If one button is slightly large, another slightly small, another large again, the page fragments into disparate elements. Amazon's home page is objectively cluttered — a trade-off forced by the need to show many SKUs — yet it feels unified because components ("related items you have viewed", "continue shopping for") share consistent sizes, and even SKU cards match each other.
- **Contrast.** Without contrast, elements disappear. Dark mode inverts copy to white; light mode inverts it back. This is why "just add a dark theme" is never a toggle: every component has to change its contrasting behaviour, and shortcuts break things. A live example: an LMS whose light theme was generated wholesale without per-component design review, leaving many components visibly out of sync.
- **Balance.** Visual weight — position, size, colour — must be distributed deliberately. One completely light component beside one completely dark component does not work.
- **Use of space.** Space must look intentional. In an LMS announcements page, a background illustration originally floated near the centre, leaving accidental gaps on both sides; pushing it flush against the right edge of the viewport turned leftover space into conscious space. ChatGPT's home screen shows the same struggle: a sparse central prompt area, chats on the left, added toggles for projects and modes — an interface still searching for its best use of space.

Examples and Case Studies: ChatGPT's navigation now separates "chat" and "work" — and the ordering is strategic. Chat comes first because 80-90 percent of users still think of LLMs as chatbots; only inside the Silicon Valley bubble has "work" become the dominant frame. Google solved for answers; LLM products are moving past answers to doing the work itself, which is why agentic products such as Claude Code and Cursor remain developer-first — the population ready to use them that way is still small. Emergent became a unicorn by collapsing the learning curve entirely: "write your idea, we build it." The same wave carried Replit, Lovable and Bolt to ARR in the 150-250 million dollar range.

Key Insight: A page is judged as a whole — unity makes it one thing, contrast makes it legible, balance makes it stable, and deliberate use of space is what separates design from accident.

7. The Fidelity-Interactivity Workflow: From Sketch to Live Product

What is it? A matrix mapping the design workflow along two axes — interactivity (y-axis) and fidelity (x-axis) — from hand sketches through clickable wireframes and mockups to the live product.

Why does it matter? Fidelity means closeness to the real product. The higher the fidelity, the less room

remains to change the product; the lower the fidelity, the broader the changes that are still cheap. Knowing where you are on this curve tells you how expensive a change of mind will be.

How does it work?

- **Hand sketches** — lowest fidelity, lowest interactivity. If a stakeholder says "this is not the design in my head", the change costs nothing.
- **Clickable wireframes** — slightly higher on both axes; rework is still modest.
- **Mockups** — real-looking designs. AI design tools such as Claude's design generation produce mockups: an announcements page generated for an LMS matched the eventual real feature to roughly 80 percent similarity — right header, right structure — while differing in the product's own design system details. Mockups can also show multiple states, such as a mobile announcements screen with and without content.
- **Live product / prototype** — highest fidelity; the back end need not exist yet. Tools like InVision historically made clickable prototypes; today Figma Make and similar tools fill this role.

The cost asymmetry is the point: at the conceptual or wireframe stage, reversing a decision is easy. Once visual design is done — colours, styles, layouts all chosen — a change forces rework across conceptual design, the wireframe, the user flow, and the visual layer all at once.

Examples and Case Studies: The mockup-generation demo showed both an admin view and a user view of the same feature, illustrating how AI tools give the 80 percent essence a designer then adapts to the product's actual design system — an exact replica is only possible when building something entirely new from scratch.

***Key Insight:** Every step up in fidelity converts flexibility into cost — make the big decisions while the design is still cheap to change.*

8. The Design Sprint: Five Days from Problem to Tested Prototype

What is it? A structured, goal-bound design exercise — often mislabelled a "design thinking workshop" — that takes a team from problem statement to user-tested prototype, classically in five days.

Why does it matter? A sprint without a goal is not a sprint; the defining property is that something shippable is shown to users at the end. Companies typically run design sprints around new product launches or critical features, and the format compresses cross-functional judgement into days. There are three types of sprints in an organisation — business sprints, design sprints, and dev sprints — and most people only ever see the dev sprint.

How does it work?

Roles:

- **Decider** — usually the VP of product or product lead; sets the objective ("we are revamping the invoicing feature") and defines the surface area, exactly as a moderator does in a user interview. Holds the super vote for tie-breaks.
- **Facilitator** — often the PM; the timekeeper and scrum-master of the sprint, pulling drifting discussions back on track.
- **Design expert** — owns the actual design and builds the prototype.
- **Engineering, marketing, sales, customer service, product** — each contributes a different definition of good design; customer service brings user feedback.

The five days:

- **Monday — decide the target.** The decider names the one problem.
- **Tuesday — sketch solutions.** Every participant, from every function, individually sketches solutions by hand. Different functions sketch differently, and that diversity is the point.
- **Wednesday — choose.** All sketches go on a bulletin board and the team votes — dot voting, speed critique, heat maps, empathy mapping are all used. Ties go to the decider's super vote.
- **Thursday — prototype.** The design expert builds a realistic prototype of the winning idea — a Goldilocks design.
- **Friday — test.** The prototype goes to beta users, alpha users or power users for feedback.

The **Goldilocks design** takes its name from astronomy: the Goldilocks zone is the orbital band with enough warmth to survive and not enough cold to freeze — the Earth-like band where life is possible (candidate worlds include Europa, a moon of Jupiter, and various planets found by the Kepler mission). A Goldilocks design is likewise in between: not so close to the production design that nothing can change, and not so close to a concept diagram that users cannot tell what is meant. AI mockup tools naturally produce designs in this zone. Sprint length flexes with feature criticality — ten days, two days, or a single morning-to-evening workshop.

Examples and Case Studies: A live exercise: five minutes to sketch a redesign of the Zoom chat window. Participants produced real ideas — highlighting the primary message in a thread, grouping emoji/attachments/raise-hand actions, a collapsible questions mode, a notes-to-self concept (which surfaced a genuine design constraint: Zoom chat evaporates when the meeting ends, unlike WhatsApp, which is likely why notes-to-self does not exist there). The deeper lesson landed immediately: it is genuinely hard to represent an idea visually in five minutes, yet PMs routinely hand designers a thought that took five minutes and demand a finished feature design in two days. A parallel example of scoped problem framing: redesigning WhatsApp for a person who receives 50-80 DMs a day — the problem statement, not the solution, is what the decider supplies.

***Key Insight:** A design sprint works because it is goal-bound and cross-functional — one decider, one problem, everyone sketches, the team votes, and a Goldilocks prototype meets real users within the same working window.*

9. The Product-Design-Engineering Workflow: Concept Note, BRD Draft and the Engineering Handshake

What is it? The correct sequencing of documents and conversations for shipping a feature: concept note, then a commentable BRD draft with design and engineering working in parallel, then an explicit handshake before dev sprints begin.

Why does it matter? The common failure is waterfall: write the whole BRD, hand it to design, design hands output to engineering, and engineering finally says "this cannot be built". Sprints get derailed precisely because engineering and design were never asked before commitments were made.

How does it work?

- **Concept note.** A one-pager — similar in spirit to a BRD — announcing to the organisation that this work is starting. It goes to design, dev, and business simultaneously.
- **BRD as a draft.** The BRD is written as a living draft with comment access for the design team and engineering leadership — not every engineer, but the engineering manager or a lead who makes

feasibility calls. Design starts working from the draft in parallel; engineering starts asking questions in the draft. For a large feature, this runs 7-10 days of daily contact.

- **Design handoff and engineering handshake.** The BRD closes only after design and engineering input is incorporated. The handshake means engineering confirmed feasibility before the document froze. Then the engineering manager sits with the PM to decompose the BRD — perhaps 200 tasks — into dev sprints of what the team can actually absorb, say 20 tasks in sprint one.
- **Realistic timelines.** Concept note to engineering handoff: about a fortnight minimum. The dev sprint: roughly another fortnight. A major feature genuinely shipped therefore takes about a month.
- **Set expectations explicitly.** When developers are involved early, they tend to ask implementation-level micro-questions too soon, assuming the PRD is final. State explicitly that the handshake has not happened yet — "ask functional questions now; detailed edge-case questions after we lock". Missed edge cases after the handshake are then legitimately the PM's to answer.
- **Run a discovery sprint in parallel.** While the real sprint executes, most PMs just watch it. Instead, the next feature's discovery should already be running in the PM's head — because a PM's job is to reduce underutilisation in the system. If engineers are busy but designers sit idle (or vice versa), that is the PM's failure. Designers in particular always end up with too little time because nobody told them early; responsive design across desktop, tablet and mobile takes real design thought, not a "wrap it in a span" instruction three days before a sprint.

Examples and Case Studies: Some B2B organisations now formalise this as "product models": product, design and engineering working together directly with customers through workshops from day one, so design understanding exists from the start. This is also why the FDE — forward deployed engineer — role is exploding. Stripped of the title, an FDE is a person sitting on-premises with the customer's design, dev and business teams, using LLMs to build the process directly. The industry motion has inverted from SaaS (software as a service) to service as a software: because LLMs have commoditised building, a person can now promise a service and deliver it as custom software. Palantir operates exactly this way, and Palantir's CEO Alex Karp has publicly argued that Anthropic and OpenAI are dangerous partners for enterprises because they take not just your money but your data — conversations about proprietary IP eventually feed foundational models. That argument is driving the open-models movement: keep your own data, run models on-premises, avoid handing IP to closed-source providers.

***Key Insight:** Ship features through a handshake, not a handoff — engineering and design must shape the BRD while it is still a draft, because feasibility discovered after commitment is how sprints die.*

10. AI-Era Prototyping: From Drawing to Prompting

What is it? The transformation of the design workflow by AI tools: the classic loop of thinking, drawing, refining, building, testing and learning becomes thinking, prompting, testing, refining and shipping.

Why does it matter? Drawing and refining — historically the slowest steps — are replaced by prompting. Describing what you want in natural language now generates UI, flows and structure; no screen needs manual redrawing; real product behaviour can be simulated; and dozens of iterations can be generated in one go. This collapses the cost of the low-fidelity-to-mockup journey described earlier.

How does it work? Prompting well is the same skill as briefing well — the PRD principles apply directly. The tool landscape has distinct personalities:

- **Figma Make** — a leading AI prototyping entrant from the design-tool world.

- **Claude's design generation** — very strong for mockups (as demonstrated with the LMS announcement pages).
- **Lovable** — very good at designing UI interfaces for websites.
- **Replit** — very good for SaaS-platform-style builds.
- **Bolt, Emergent** — same category; Emergent's pitch is that users need not think about any of the machinery: write the idea, receive the build.

Why these platforms have such distinct design philosophies only becomes clear with an understanding of LLMs — each is built on models and scaffolding with different biases, which is exactly the subject of the LLM block below. These platforms also have pitfalls, which are addressed once the fundamentals are in place.

Examples and Case Studies: The startups riding this shift — Replit, Lovable, Bolt, Emergent — reached the 150-250 million dollar ARR range because they bet that most people would never climb the learning curve of developer-first tools. Emergent's unicorn status is a direct monetisation of the emergent capabilities of LLMs described later in these notes.

***Key Insight:** Prompting has replaced drawing in the design loop — which makes the ability to describe a product precisely in natural language a core design skill, not a technical one.*

11. From Analytics to Intelligence: Turing's Question and the Three Analytics

What is it? The historical and conceptual foundation of AI: Alan Turing's 1950 question "Can machines think?", the distinction between knowledge and intelligence, and the three families of analytics that preceded the LLM era.

Why does it matter? Knowledge and intelligence are routinely conflated, but they are not the same: intelligence is knowledge plus application — the ability to actually do something with what is known. LLMs matter because for the first time a non-human can pick up a task and complete it: ask for a PPT and tools like Gamma simply make it. That is an intelligence explosion, not merely a knowledge explosion.

How does it work? Before LLMs, humanity's analytical machinery had three layers:

- **Descriptive analytics** — describing the past. If a salary was Rs 1 lakh in 2020 and Rs 2 lakh in 2024, descriptive analytics states the 100 percent growth. It cannot say anything about 2027. This is the domain of business analysts, data analysts and MIS analysts.
- **Predictive analytics** — projecting forward. Fit a trend line and estimate the 2027 salary at roughly Rs 2.5 lakh — possibly right, possibly wrong. This is the domain of data scientists, supported by data engineers who build the pipelines and infrastructure (technologies such as Kafka and Spark for distributed big-data systems). The frontier now includes on-device local models that run predictions on a phone even with the internet off.
- **Prescriptive analytics** — deciding what to do. This is the product manager's layer: combining descriptive evidence ("20 percent discounts lifted sales before") with predictive models ("the model projects a 50 percent sales increase") to prescribe an action, such as shipping a discounting feature.

Terminology that must not be blurred: data science is the superset containing descriptive plus predictive analytics — any analysis of data for insight. Machine learning, neural networks, deep learning and artificial intelligence all mean different things, and the next topic defines them precisely.

Examples and Case Studies: Alan Turing — the subject of the film *The Imitation Game* — posed "Can machines think?" in 1950, the founding question of the field. The question stayed philosophical for seventy years because the machinery to attempt an answer did not exist.

***Key Insight:** Intelligence is knowledge plus application — analytics could describe and predict, but the LLM era is the first time machines act on knowledge, which is what makes this moment categorically different.*

12. Machine Learning and Neural Networks: Weights, Backpropagation and the AI Winter

What is it? The formal definition of machine learning, the architecture of neural networks, and the two missing ingredients — data and compute — whose arrival ended the AI winter.

Why does it matter? LLMs are not magic; they are the scaled-up descendants of a simple idea — systems that reduce their own error with every iteration. Understanding the mechanism strips away both the hype and the fear.

How does it work?

- **Machine learning defined.** Any system where output feedback improves future outputs: $y = f(x) + e$, where y is the output (say, salary), $f(x)$ is a function of inputs (years of experience), and e is error. If the error shrinks with every iteration, that is machine learning. It does not require an LLM — a rule-based system that feeds its errors back qualifies, and even the humble calculator was an early example of machines improving computation. Machine learning sits inside artificial intelligence.
- **Neural networks.** In the 1970s, researchers including Geoffrey Hinton — called one of the godfathers of AI, alongside Yann LeCun and others — proposed systems modelled on the human brain's neurons. A neural network is layers of nodes: input nodes, processing nodes, output nodes, each node a processing unit that can run algorithms (linear regression, logistic regression, support vector machines, random forests, XGBoost and others). Each connection carries a **weight**: predicting salary might combine $a_1 \times \text{years-of-experience} + a_2 \times \text{title} + a_3 \times \text{pay-band}$, where a_1 , a_2 , a_3 are unknown weights.
- **Backpropagation.** The network's first prediction is wrong — it predicts Rs 2,20,000 when the true salary is Rs 2,25,000. The error is propagated backwards through the network, nudging every weight so the next prediction lands closer. Iterated millions of times, the network converges. The human analogy: a first PRD is useless, feedback adjusts the internal weights, the second is better, the third better still — the same cycle of error correction building better internal connections.
- **The AI winter (1970-2010).** For four decades, working on neural networks was career suicide, for two reasons: no devices could run them, and no data existed to feed them. Both constraints then broke. Data became abundant when Jio collapsed data prices — and the effect was global, not just Indian, as worldwide pricing followed. (Fact-check: Jio launched commercially in September 2016, though the cheap-data era is loosely dated from the early 2010s smartphone boom.) Compute arrived through NVIDIA under Jensen Huang — now among the world's richest individuals — whose CUDA parallel-processing framework let enormous volumes of data be processed fast, and whose hardware went early to a then-tiny startup called OpenAI. (Fact-check: CUDA was released in 2006-07, not 2017; NVIDIA hand-delivered its first DGX-1 AI supercomputer to OpenAI in 2016.) With data and compute finally available, the path to large language models opened.

Examples and Case Studies: The salary-prediction network is the canonical classroom example: inputs flow through weighted nodes like ten people standing in a line, each processing and passing an answer

onward, with the final error flowing back down the line to make everyone better. Engineering questions — how a failed node is handled, how robustness is ensured — are real but belong to the mathematics (matrix algebra, transposes, linear algebra) beneath this conceptual layer.

Key Insight: All of modern AI reduces to one loop — predict, measure the error, propagate it back, adjust the weights — and it only became world-changing when cheap data and parallel compute finally let the loop run at scale.

13. Large Language Models: Scale, Scaling Laws and the Three Training Stages

What is it? What an LLM actually is — a neural network of extraordinary scale — plus the economics and philosophy of model size, and the three stages of training: pre-training, fine-tuning, and post-training.

Why does it matter? Product managers now interface with this technology daily. Knowing what is happening under the hood is the difference between using recipes and making architecture-level product decisions — which model, what error tolerance, what cost profile.

How does it work?

- **Scale.** An LLM is a neural net with an enormous number of parameters. Early GPT-generation models had tens of millions of parameters (fact-check: GPT-1 had about 117 million parameters; GPT-2 scaled to 1.5 billion). Frontier models today — the session cited Claude's current Fable 5 model at a reported 7 trillion parameters — are roughly five orders of magnitude larger, and still return answers in seconds, which is a staggering feat of engineering.
- **Scaling laws.** The bigger the model, the better and more general the output — a relationship that has held so far, with no ceiling yet hit. This is why labs like Anthropic keep building larger models: size buys generalisation, meaning more skills. It also explains specialisation: Claude's models are heavily biased towards coding data and excel there, ship no image model, and write weaker copy; Gemini writes better copy but generates weaker code.
- **The cost-efficiency counter-philosophy.** Chinese labs (DeepSeek, Kimi) optimise for token efficiency rather than maximum size: roughly 95 percent of frontier quality at a fraction of the cost. Techniques include **mixture of experts** — a model may hold trillions of parameters but activate only, say, 50 billion for a given query, routing work to the nodes expert in that domain — and **distillation**, compressing a large model into a much smaller one with only a few percentage points of quality loss (whether the distilled-from models were used with permission is, for classroom purposes, "alleged"). The knowledge paradigm behind distillation is essentially Pareto: 80-90 percent of the knowledge yields 95 percent of the output. Diffusion models — a different architecture entirely — dominate image generation.
- **Pre-training.** The most expensive stage, already done by the big labs (Anthropic, OpenAI, xAI, DeepSeek, Kimi): training on petabytes scraped from across the internet — full Wikipedia dumps, Reddit's archives (Reddit later locked its data behind a paid API and now charges for training access). Pre-training gives the model its generalised capabilities — emails, presentations, every broad skill. It also fixes the **knowledge cutoff**: before web search existed as a tool, a model asked about yesterday's cricket century would answer from data months old, because its knowledge ended at its last training run. Tools now let models fetch current information live.
- **Fine-tuning.** Taking a pre-trained model and specialising it on proprietary data. A live demonstration: a custom GPT trained on an educator's own corpus, asked "what is product sense?", searched only its uploaded knowledge and answered in the educator's exact framing — product sense as the ability to look at a messy situation, figure out who has a real problem and why,

and identify the simplest useful thing to build; the micro and macro interactions that improve a user's life. Such a model can either fall back to pre-trained knowledge or be given a hard cutoff — "if it is not in my data, say I don't know." Most applied industry work happens at this fine-tuning layer.

- **Post-training.** The third stage (including techniques such as reinforcement learning), deferred to a later session.
- **SLM vs LLM as a product decision.** Small language models can run on-device. Choosing between them is an error-rate call: for apparel commerce, a 2 percent error rate may beat human return rates and is acceptable; for a pharmacy order, delivering the wrong medicine is intolerable — the false-negative tolerance of the product decides the model.

Examples and Case Studies: The specialisation trade-off is directly product-relevant: a team building for a niche domain should expect general models to be weak there and consider fine-tuned or custom architectures — which is exactly the context behind the transformer discussion later.

***Key Insight:** An LLM is a neural net scaled to trillions of parameters, and every practical choice — model, cost, quality, error tolerance — flows from three levers: how big it is, what it was trained on, and how much of it you activate per query.*

14. Emergent Behaviour and Generative AI

What is it? Emergent behaviour — capabilities a model was never explicitly trained for — and the composition of LLMs with image and video models that the market calls generative AI.

Why does it matter? Emergent behaviour is the property that made the world sit up: the first time in history that a trained system demonstrably generalised beyond its training objective. It is why LLMs feel intelligent, and it is literally the namesake of the unicorn startup Emergent.

How does it work?

- **Emergence.** Train a person to box strictly for self-defence, and they can nonetheless box anywhere — the skill generalises beyond the training setting. The same happened with LLMs: a model trained to translate English to German turned out to translate English to French, and English to Hindi, without ever being told to. Nobody instructed it; the capability emerged from scale.
- **Interpretation of vague queries.** Emergence also shows up as the model understanding what you meant. Asked "who is the greatest director of all time?" — with an expectation of an answer like Stanley Kubrick from a questioner who had been reading about Steven Spielberg and Martin Scorsese — a model answered Akira Kurosawa, the Japanese master, having inferred that the question was not scoped to American directors. The model filled in the intent behind the underspecified question.
- **Generative AI is a composition.** A large language model outputs only text — never anything else. The images and videos in ChatGPT come from pairing: the LLM writes a prompt, hands it to a separate image model (or video model), and the paired system returns the picture. This pairing is what the umbrella term generative AI actually denotes. Dedicated video-model companies include Kling and ByteDance (of TikTok), whose recent model is Seedance 2. This is also why people sell "image prompts" — the text the LLM feeds the image model is itself the craft.

Examples and Case Studies: The two-model architecture is invisible to users — one prompt, one picture — but product-critical: latency, cost and quality of an image feature depend on a pipeline of models, not one.

Key Insight: LLMs earned the label "intelligent" the day they did things nobody trained them to do — and everything visual they appear to produce is actually a second model taking dictation from the text model.

15. Tokens: The Currency of LLMs

What is it? Tokens — the units in which LLMs read, process, price and generate everything.

Why does it matter? You pay for tokens, in both money and context capacity. Understanding tokenisation converts directly into cheaper, more reliable prompting.

How does it work?

- **A token is a vectorised notation.** A word is thrown into a multi-dimensional space and represented as coordinates — a graph position, not a meaningful number. The token ID is a variable, an arbitrary code standing in for the word, the way a jersey number identifies a player without being the player's value (fact-check: the jersey example cited Sachin Tendulkar as 7; Tendulkar's famous ODI number was 10 — 7 is MS Dhoni's).
- **Roughly four characters per token.** A tokenizer demo showed a 24-character sentence becoming 7 tokens; a common short word is typically a single token. This is why tokens and words get used interchangeably — approximately, not exactly.
- **Training-set presence drives token cost.** Common words tokenize efficiently because pre-training built rich relationships around them; rare or archaic words split into more tokens. It is not the difficulty of the word but its presence in the training data: a well-known Latin phrase tokenizes cheaply, an extinct one does not. "Stop firing" took 2 tokens where "cease firing" took 3. Ornate vocabulary literally costs more money per prompt.
- **Speak the model's vocabulary.** Describing a UI element as "the right button on the right screen, slightly bigger, red in colour" sends Claude Code hunting; saying "it's an accordion" resolves instantly, because "accordion" is a dense, well-trained term. This is the same reason product managers share vocabulary like "orthogonal" — precise shared terms compress communication. Common vocabulary with the LLM is token efficiency.
- **Self-built relationships.** Unlike a database, where relationships must be defined by rules, an LLM builds relationships automatically. Feed it three sentences about a person — a great cricketer, a book lover — and ask about hobbies: it infers cricket, though hobbies were never mentioned. One paragraph about a person lets it answer what she likes, wears, and where she lives, by traversing the relationship graph it built itself. This self-assembling relational structure is the source of LLM power.
- **Token spaces differ by model.** Each model builds its own token space, so the same word maps to different IDs in ChatGPT and Claude — though within one model the mapping is consistent. Input tokens and output tokens are also counted (and priced) differently. Identical prompts to two models yield different answers because token space and model design differ: perhaps 90 percent of the factual content matches, but the structure and framing diverge, the way two knowledgeable people give the same answer in their own words.
- **Token cost scales with model and reasoning.** More powerful models and higher reasoning levels cost more per token — an expert consultant charges thousands per hour where an intern charges a fraction, and "think deeply, then answer" is the expensive instruction.

Examples and Case Studies: The tokenizer demonstration (word counts, character counts, token IDs shown live) is reproducible on any public tokenizer tool and is worth repeating on your own prompts to

build intuition for what your requests actually cost.

Key Insight: Tokens are money and meaning at once — prompt in the plain, well-trodden vocabulary the model was trained on, and you get answers that are simultaneously cheaper and more accurate.

16. Prompting, Context and Hallucination

What is it? The working rules for giving LLMs the right amount of information — and the failure mode, hallucination, when they are given too little.

Why does it matter? "Prompts should be very detailed" is bad advice. Prompts should be optimal: too vague and the model assumes; too long and it drowns — the same lost-in-translation physics as a PRD.

How does it work?

- **The optimal-length principle.** A two-minute conversation conveys nothing; after a two-hour conversation, the other person cannot recall how it started; a fifteen-minute conversation is fully remembered. Prompts behave identically — under-size and over-size both degrade output.
- **The obviousness rule.** Something obvious to a human is obvious to the LLM; something not obvious to a human is not obvious to the LLM. "Wrap the text in the button" needs no elaboration — the model knows. A custom animation unlike anything standard must be specified in detail. Albus Dumbledore needs no introduction — the term saturates the training data; a grandmother's unique recipe, or a family deity ritual known only to one household, must be fully described or the model will produce something vague and wrong.
- **Repetitive tasks need thin prompts.** For routine, well-understood operations, terse instructions work; save the detailed specification for the genuinely novel.
- **Hallucination.** LLMs are designed to give answers, not to question the asker. Given an underspecified or unknowable request, the model will confidently produce something anyway — and assert it is correct. This is a structural property, not an occasional bug. The practical defence in code workflows: explicit verification gates, written into the working instructions, because the model will claim "everything is right, everything will work" right up until a user reports that it does not. Never accept a completion claim without verification.

Examples and Case Studies: The deployment anecdote: code declared perfect by the model, deployed, and immediately found broken by the first user — the exact scenario verification gates exist to prevent.

Key Insight: An LLM answers confidently whether or not it knows — so give it optimally sized context, describe only what is genuinely non-obvious, and never accept its own word that the work is correct.

17. Do LLMs Think? Interpretability and the Reasoning Loop

What is it? The honest state of the "can machines think" debate, the interpretability research that peers inside models, and the mechanical meaning of reasoning levels.

Why does it matter? Product managers configure reasoning levels and pay for them. Knowing what reasoning actually is — and what thinking-like behaviour models exhibit — grounds those decisions in mechanism rather than marketing.

How does it work?

- **The debate.** Two camps exist: Anthropic-aligned voices insist models can think; opposing researchers insist they cannot. The line is genuinely thin — akin to asking at what moment consciousness arose, or when a single-celled organism became multi-celled. Edsger Dijkstra's famous line frames the scepticism: asking whether a machine can think is about as interesting as asking whether a submarine can swim.
- **Interpretability.** A research field dedicated to what models do inside — which is genuinely not fully known. Recent Anthropic interpretability work shows internal processing that resembles human cognition: asked "who is the prime minister of India?", the model internally activates prime-number associations at the word "prime", suppresses them upon reading "minister", surfaces candidate faces — a UK prime minister, Modi — and resolves to an answer. None of this internal deliberation is shown to the user. It mirrors a person silently generating three or four interpretations of a question before answering. The honest verdict: whether models truly think is unresolved, but they demonstrably resemble and mimic a great deal of thinking behaviour.
- **The reasoning loop.** Reasoning is mechanically simple: define the requirement, do the requirement, verify the requirement — then loop. The reasoning level sets the loop count. Asked to design a website, a model at low reasoning might run 5 verification steps; at maximum reasoning, perhaps 25, verifying every step. More loops mean better-checked output and higher cost — and even at maximum, wrong answers remain possible. The model's stopping rule works the same way: as output tokens generate, a verification mechanism checks them against patterns from the pre-training set (generated code, for instance, is checked against parallel boilerplates) until the checks pass and the response completes.

Examples and Case Studies: The generated-code case makes the mechanism concrete: the model verifies its output against the code patterns it has seen, which is why its self-verification can pass while the code still fails in your specific context — its reference set is the internet's code, not your system.

***Key Insight:** Reasoning is a purchasable quantity of define-do-verify loops — models mimic thinking convincingly and check their own work statistically, which is powerful, and still no substitute for your verification.*

18. Transformers and Attention: The Architecture Behind It All

What is it? The transformer — the architectural design introduced in Google's 2017 paper "Attention Is All You Need" — that underlies effectively every modern LLM.

Why does it matter? The paper is one of the most cited in the history of computer science, and the entire AI revolution followed from it. When advisors tell a team "build your own transformer model", this is what they mean — and a PM should understand the suggestion.

How does it work?

- **The memory problem.** Classical and heuristic computer science struggled with memory retention across long sequences. The attention architecture gave models effectively unlimited memory — the ability to retain and reference anything in context, the way a person can still recall a vivid childhood event decades later. Whether a given memory is useful is a separate question from whether it is retained.
- **The lineage.** Earlier architectures included BERT-style models (bidirectional encoding). Transformers comprise encoder and decoder components, and every major lab now runs its own modified version — OpenAI's variant, Anthropic's rewrite — but the core paper came from Google.

Novel experimental designs in this layer (memory-efficient variants, context-preserving variants) are exactly where frontier research competes.

- **Custom transformers.** For a niche domain — geostatistics, for example, where general models have thin training data — building a domain-optimised transformer architecture means designing a model whose architecture is tuned to that use case, rather than fine-tuning a general one.
- **Staying current.** This knowledge changes every ten days. No book can carry it: staying on the frontier means Twitter, Substack and arXiv research papers, continually. Courses on AI hold value only when the person teaching is actively building with the technology — six-month-old information becomes irrelevant within six months. For the underlying fundamentals, the mathematics-heavy but definitive references are Ian Goodfellow's Deep Learning and Sebastian Raschka's machine learning books — both requiring comfort with linear algebra (if the transpose of a matrix means nothing, the maths track is not yet open).

Examples and Case Studies: The rationale for teaching theory at all, in a program aimed at application: most people jump straight to application and plateau, able to follow recipes ("these 5 skills will humanize your text") without a functional understanding of why anything works. With the fundamentals in place, six months of self-directed work compounds upward instead of requiring a return to basics — the program defines the baseline; each learner defines the ceiling.

***Key Insight:** One 2017 paper — Attention Is All You Need — gave models unlimited memory and launched the entire LLM era; the fundamentals compound forever, but the frontier must be read fresh every ten days.*

Frameworks and Mental Models

- **Three Metric Lenses (Business / Product / Marketing).** Every product system can be read through business metrics (revenue, cost, margin, EBITDA), product metrics (retention, churn, activation, funnels), and marketing metrics (CAC, LTV, CPI). Use it when evaluating any product, writing a BRD, or diagnosing whether growth is real; a healthy product satisfies all three lenses at once. Visualise three overlapping views of a single system — like inspecting one house through budget, structure and livability.
- **The A-by-B Metric Rule.** Any legitimate metric takes the form A divided by B (revenue per user, retention per month), because ratios strip out bias that absolute numbers hide. Use it to test every metric in a document or resume: if there is no denominator, the metric is faff.
- **Thorough-Succinct Balance (the Orbit Model).** A PRD must be thorough enough to carry the full essence and succinct enough to be read — two opposing forces cancelling like gravity and centrifugal force keeping a satellite in orbit. Use it whenever writing PRDs (18-25 pages maximum) or prompts; imbalance in either direction produces lost-in-translation failures and scope creep.
- **Proximity and Similarity Principles.** Elements close together with shared visual traits read as equally important; elements that look alike must behave alike. Use them when reviewing any interface: check that visual emphasis matches actual priority and that lookalike components never diverge in function.
- **Composition Checklist (Unity / Contrast / Balance / Space).** Four page-level tests: does it read as one page, is everything legible, is visual weight distributed, is empty space intentional? Use it as a structured design-review pass before any handoff.
- **The Fidelity-Interactivity Matrix.** A two-axis map (interactivity vertical, fidelity horizontal) running from hand sketches through clickable wireframes and mockups to live product. Use it to decide

when to lock decisions: cost of change rises with every step right, so contest direction early and cheaply. Visual: a diagonal staircase where each step trades flexibility for realism.

- **The Five-Day Design Sprint.** Monday decide the target, Tuesday sketch solutions individually, Wednesday vote (dot voting, heat maps, empathy mapping; decider holds the super vote), Thursday build a Goldilocks prototype, Friday test with beta or power users. Use it for critical features and new launches; compress or stretch the calendar to match criticality.
- **The Goldilocks Design.** A prototype pitched between production-fidelity (too rigid to change) and concept-fidelity (too abstract to test) — named for astronomy's habitable zone. Use it as the target artefact of any design sprint or AI mockup pass.
- **Concept Note → BRD Draft → Engineering Handshake.** A one-page concept note announces the work; the BRD lives as a commentable draft while design works and engineering questions in parallel; the handshake locks feasibility before sprint decomposition (about a fortnight to handoff, another to ship). Use it for every non-trivial feature; the anti-pattern is the waterfall handoff where engineering sees the plan last.
- **Discovery Sprint / Delivery Sprint Parallelism.** While the delivery sprint executes, the PM's discovery sprint for the next feature must already be running — because the PM's job is to reduce underutilisation across engineering and design. Use it to plan designer lead time, which is the resource most consistently starved.
- **Descriptive / Predictive / Prescriptive Analytics.** Describe the past, project the future, decide the action — owned respectively by analysts, data scientists, and product managers. Use it to locate any data conversation and to know which discipline (and which evidence) a decision requires.
- **The Machine Learning Loop ($y = f(x) + e$).** Output equals a weighted function of inputs plus error; learning is shrinking the error every iteration via backpropagation. Use it as the mental model for all AI — and for human skill acquisition, from PRD writing to design judgement.
- **Pre-training / Fine-tuning / Post-training.** The three training stages: generalised capability from internet-scale data (done by big labs), specialisation on proprietary data (where most applied work happens), and post-training refinement. Use it to place any AI product decision — most PM-adjacent work lives at the fine-tuning layer.
- **The Define-Do-Verify Reasoning Loop.** A model's reasoning level is the number of define-do-verify cycles it runs before responding (roughly 5 at low, 25 at maximum). Use it to choose reasoning levels deliberately: more loops buy reliability at higher token cost.
- **The Obviousness Rule for Prompting.** What is obvious to a human is obvious to the LLM; what is not, is not. Use it to size every prompt: name standard things tersely ("it's an accordion"), specify unique things exhaustively.

Key Terminology

- **Copy:** The text on a product or marketing surface; the words that carry meaning alongside visuals.
- **Connotation:** The emotional register in which a word is used, distinct from its literal meaning.
- **FTUE (First Time User Experience):** The guided walkthrough shown to a new user explaining what each feature does.
- **Responsive design:** Design that remains visible, valuable and usable across all screen sizes and device types.
- **Accessibility:** The dimension of UX ensuring every intended user can easily reach the product's value.
- **Fidelity:** How close a design artefact is to the real product; high fidelity means near-final, low

fidelity means sketch-level.

- **Wireframe:** A low-fidelity structural outline of a screen, optionally clickable.
- **Mockup:** A realistic-looking design image of a product screen, without working functionality.
- **Prototype:** An interactive simulation of the product, possibly without any back end.
- **Design sprint:** A goal-bound, time-boxed cross-functional exercise producing a user-tested prototype.
- **Decider:** The design-sprint role (usually VP product) that sets the objective and holds the super vote.
- **Facilitator:** The design-sprint timekeeper — effectively its scrum master, usually the PM.
- **Dot voting:** Selecting among sketched solutions by having participants place votes on a board.
- **Super vote:** The decider's tie-breaking or overriding vote in a design sprint.
- **Goldilocks design:** A prototype between concept and production fidelity — real enough to test, loose enough to change.
- **Concept note:** A one-page document announcing to the organisation that work on an initiative is starting.
- **BRD (Business Requirements Document):** A short document of business metrics establishing why an initiative is viable.
- **PRD (Product Requirements Document):** The culminating requirements document; must be thorough yet succinct.
- **Engineering handshake:** Explicit engineering feasibility sign-off on a BRD draft before it is locked and sprints begin.
- **Scope creep:** Uncontrolled expansion of implementation beyond intent, typically caused by overlong documents or prompts.
- **Sprint:** A time-boxed execution cycle with a shippable goal; business, design and dev variants exist.
- **FDE (Forward Deployed Engineer):** An engineer embedded on-premises with a customer's business, design and dev teams, building solutions directly using LLMs.
- **Service as a software:** The inversion of SaaS — promising a service and delivering it as custom-built software, enabled by LLM-commoditised building.
- **COGS:** Cost of goods sold — the direct cost of producing what a business sells.
- **EBITDA:** Earnings before interest, taxes, depreciation and amortization; the A covers depreciation of intangible assets.
- **Amortization:** The scheduled reduction in value of intangible assets over time.
- **Contribution margin (CM1/CM2/CM3):** Layered margin measures after successive tranches of variable costs.
- **CAC (Customer Acquisition Cost):** Total acquisition spend divided by paying customers acquired.
- **LTV (Lifetime Value):** Total value a customer generates over their relationship with the product.
- **Lead:** A potential customer who has shown interest but not paid; distinct from a customer.
- **Retention:** The fraction of users who return, always qualified by a time denominator (daily, monthly, yearly).
- **Churn:** The rate at which users or subscribers leave a product.
- **Descriptive analytics:** Analysis describing what happened in past data.

- **Predictive analytics:** Modelling that projects what will happen from past data.
- **Prescriptive analytics:** Deciding what action to take, combining descriptive and predictive evidence; the PM's layer.
- **Data science:** The superset of descriptive and predictive analytics — any data analysis producing insight.
- **Machine learning:** Any system whose outputs improve through iterative error feedback; $y = f(x) + e$ with shrinking e .
- **Neural network:** Layered nodes with weighted connections, modelled loosely on brain neurons, that learn by adjusting weights.
- **Node:** A single processing unit in a neural network, capable of running algorithms internally.
- **Weight:** A tunable coefficient on a network connection, adjusted during training.
- **Backpropagation:** Feeding output error backwards through a network to adjust weights and reduce future error.
- **AI winter:** The period (roughly 1970-2010) when neural network research stalled for lack of data and compute.
- **CUDA:** NVIDIA's parallel-processing framework enabling fast computation over high volumes of data.
- **LLM (Large Language Model):** A text-only neural network with an enormous parameter count, trained on internet-scale data.
- **SLM (Small Language Model):** A compact language model, often capable of running on-device.
- **Parameter:** A learned weight in a model; parameter count is the standard measure of model size.
- **Scaling laws:** The observed relationship that larger models produce better, more general outputs — holding so far without a ceiling.
- **Mixture of experts:** An architecture that activates only a relevant subset of a model's parameters per query.
- **Distillation:** Compressing a large model into a much smaller one with minimal quality loss.
- **Diffusion model:** A model architecture particularly suited to image generation.
- **Pre-training:** The expensive first training stage on internet-scale data, granting generalised capabilities.
- **Fine-tuning:** Specialising a pre-trained model on proprietary or domain data.
- **Post-training:** The refinement stage after fine-tuning, including techniques such as reinforcement learning.
- **Knowledge cutoff:** The date of a model's last training run, beyond which it knows nothing without tools.
- **Tool (LLM context):** An external capability, such as web search, that a model can invoke to fetch live information.
- **Emergent behaviour:** Capabilities a model exhibits that it was never explicitly trained for.
- **Generative AI:** The pairing of an LLM with image or video models to produce non-text output.
- **Token:** The unit of LLM processing — a vectorised representation of a text fragment, roughly four characters on average.
- **Tokenizer:** A tool that converts text into tokens and shows the resulting token IDs and counts.
- **Token space:** A model's specific mapping of text to tokens; differs across models.

- **Context:** The information available to a model within a conversation or prompt.
- **Hallucination:** A model confidently producing incorrect output, a structural consequence of being designed to answer rather than question.
- **Interpretability:** The research field studying what models actually do internally.
- **Reasoning (level):** The number of define-do-verify loops a model runs before responding; higher levels cost more and check more.
- **Transformer:** The 2017 attention-based architecture underlying modern LLMs, comprising encoders and decoders.
- **Attention:** The mechanism from "Attention Is All You Need" giving models effectively unlimited memory over context.
- **BERT:** An earlier bidirectional-encoding model family predating decoder-style LLMs.
- **NPA (Non-Performing Asset):** A loan on which repayment has stopped; the core risk any lending model must underwrite against.
- **TAM (Total Addressable Market):** The full market size for a product; a superset that must be sub-segmented to be meaningful.

Action Items and Homework

1. **Elder care case study (primary assignment).** Building for India's elder care market. Context: India has 157 million people aged 60 and above, projected to reach 346 million by 2050; the silver economy is estimated at USD 7 billion with significant headroom across healthcare, housing and finance; the senior living market is projected to grow 300 percent to USD 7.7 billion; nearly 19,500 Indians turn 60 every day while younger generations migrate for work. The market has no shortage of products — it has a shortage of clarity. The leadership brief is not to build a product but to answer: what is the most important problem that remains unsolved in India's elder care ecosystem? Required work: understand the ecosystem; identify all stakeholders (elderly parents value independence, dignity, ease of access and trust; adult children value peace of mind and visibility; doctors value continuity; caregivers struggle with coordination and scheduling); conduct primary research; map the complete care journey; identify unmet needs; prioritise one opportunity; estimate the business opportunity; recommend an MVP; define success metrics.
 - **Deliverable:** A structured document of at most 15-17 pages covering the problem space first and then — mandatory this time — the solution space. A prototype is good-to-have.
 - **Approach:** Discover, converge; then for solutions, diverge and converge again. Do not begin with the solution.
 - **Evaluation:** Discovery 15-25 points, opportunity 10 points, solution thinking 30 points, communication 15 points. The strongest submissions will demonstrate deep ecosystem understanding, identification of the right problem, and justification of why it deserves solving — not the most ambitious proposal.
 - **Deadline:** 11:59 PM, 1 August.
2. **Practice the tokenizer.** Reproduce the tokenizer demonstration on your own prompts to build intuition for token counts and cost.
3. **Apply visual design principles to live products.** When looking at any design from now on, consciously read proximity, similarity, unity, contrast, balance and use of space.
4. **Bring discovery into your organisation.** Carry the discovery practice from the program into workplace decisions, even where the organisation does not currently do it.

5. **Optional deep-dive vote (implicit).** Signal whether upcoming sessions should include a dedicated two-hour fundamentals-of-LLMs lecture or move straight to applied work on Claude Code and Codex; separate hands-on sessions are available for those who want the deeper track.
6. **Prepare for the applied AI track.** Upcoming sessions cover the agentic paradigm — context, memory (short-term and long-term), skills, sub-agents, models and reasoning — building on today's fundamentals.

Resources Mentioned in the Session

Books

- **Deep Learning — Ian Goodfellow** (with Yoshua Bengio and Aaron Courville): the definitive mathematics-forward reference for neural network fundamentals. (The session's transcript garbled the name; the author is Ian Goodfellow.)
- **Sebastian Raschka's machine learning books** (e.g. Machine Learning with PyTorch and Scikit-Learn; Build a Large Language Model (From Scratch)): rigorous, maths-inclusive treatments of ML and LLMs. (The transcript rendered the surname incorrectly; the author is Sebastian Raschka.)
- Both require comfort with linear algebra and matrix operations to be useful.

Philosophical and literary references

- **"Computing Machinery and Intelligence" — Alan Turing (1950):** the paper posing "Can machines think?", the founding question of AI. Turing's life is dramatised in the film The Imitation Game.
- **Edsger Dijkstra's submarine aphorism:** the question of whether machines can think is about as interesting as the question of whether submarines can swim — the canonical sceptical framing of machine intelligence.
- **"Attention Is All You Need" — Vaswani et al., Google (2017):** the transformer paper; among the most cited papers in computer science history and the architectural origin of modern LLMs.
- **Harry Potter (Albus Dumbledore):** used to illustrate that terms saturating the training data need no explanation in prompts.
- **The Goldilocks zone:** astronomy's habitable-zone concept, borrowed to name the ideal prototype fidelity. (Correction: the session placed Europa around Saturn; Europa is a moon of Jupiter. Candidate habitable exoplanets come from the Kepler mission.)

People and public figures referenced

- **Alan Turing** — mathematician; posed the machine-thinking question in 1950.
- **Geoffrey Hinton** — a godfather of AI; pioneer of neural networks from the 1970s onward.
- **Yann LeCun** — fellow deep-learning pioneer alongside Hinton.
- **Jensen Huang** — NVIDIA founder-CEO; CUDA and NVIDIA hardware supplied the compute that ended the AI winter. (Correction: CUDA shipped in 2006-07, not 2017; NVIDIA delivered its first DGX-1 supercomputer to OpenAI in 2016.)
- **Mukesh Ambani** — Reliance Jio's cheap data collapsed data prices and created the data abundance neural networks needed. (Correction: Jio launched commercially in September 2016, not 2010.)

- **Alex Karp** — Palantir CEO; argues that closed-model providers such as Anthropic and OpenAI take enterprises' data and IP along with their money, fuelling the open-models movement.
- **Edsger Dijkstra** — computer scientist; source of the submarine-swimming aphorism.
- **Akira Kurosawa, Stanley Kubrick, Steven Spielberg, Martin Scorsese** — directors cited in the emergent-behaviour example of an LLM inferring unstated question scope.
- **Sachin Tendulkar** — cited in the jersey-number analogy for token IDs. (Correction: the session gave Tendulkar's number as 7; his famous ODI jersey number was 10 — 7 belongs to MS Dhoni.)
- **Shashi Tharoor** — shorthand for ornate vocabulary, which costs more tokens.

Products and tools referenced

- **Amazon, Flipkart, Myntra** — e-commerce interfaces used to demonstrate proximity, similarity, unity and copy principles.
- **Zepto** — quick-commerce example for copy register ("buy", never "enroll").
- **Linear, Notion, Vercel** — world-class design references for B2B and deep-tech SaaS dashboards.
- **Apple, Duolingo** — design-principle references for consumer products.
- **OpenRouter** — LLM routing platform; example of audience-calibrated (deep-tech developer) UX and copy.
- **ChatGPT / OpenAI** — home-screen composition example; chat-vs-work navigation; custom GPTs for fine-tuned corpora; GPT-1/GPT-2 as scale reference points. (Correction: GPT-1 had about 117 million parameters, GPT-2 1.5 billion — the session cited 65 million.)
- **Claude / Anthropic** — coding-biased frontier models; scaling-laws philosophy; interpretability research; design/mockup generation; Fable 5 cited at a reported 7 trillion parameters.
- **Claude Code, Cursor** — developer-first agentic coding tools.
- **Codex** — the alternative applied-AI track alongside Claude Code for upcoming sessions.
- **Gemini (Google)** — stronger at copywriting, weaker at code generation, in the model-specialisation comparison.
- **DeepSeek, Kimi (Kimi K3), xAI/Grok** — frontier and cost-efficiency-focused labs; Chinese labs as the token-efficiency counter-philosophy.
- **Figma Make, Lovable, Bolt, Replit, Emergent** — AI prototyping and app-generation tools; Lovable excels at website UI, Replit at SaaS builds; Emergent reached unicorn status; the category sits in the 150-250 million dollar ARR range.
- **InVision** — the historical clickable-prototype tool preceding today's AI options.
- **Gamma** — AI presentation generation, an example of task-completing intelligence.
- **Kling, ByteDance (Seedance 2)** — video-generation model companies; TikTok's parent ByteDance ships Seedance 2.
- **NVIDIA CUDA** — the parallel-processing framework behind the compute revolution.
- **Kafka, Spark** — big-data technologies data engineers use for distributed pipelines.
- **Jio** — the cheap-data catalyst for AI-scale data abundance.
- **Palantir** — the archetype of the service-as-a-software and on-premises model.
- **Reddit, Wikipedia** — canonical pre-training corpora; Reddit now licenses its data behind a paid API.
- **MyGate-style society-management apps, Swiggy hubs** — referenced as existing partial solutions illustrating why competitor analysis must precede problem sizing.

- **Zoom, WhatsApp** — subjects of the live redesign sketching exercise and the prompt-length analogy.
- **Shark Tank** — illustration of research-driven credibility with investors.

Community and ongoing learning

- **Twitter, Substack, arXiv** — the only viable sources for frontier LLM knowledge, which changes every ten days; daily reading of research papers and practitioner writing is the recommended discipline.
- **LinkedIn** — research tool for understanding interviewers and investors before high-stakes conversations.
- **Optional deep-dive sessions** — a two-hour LLM fundamentals lecture and separate hands-on sessions are on offer for those who want the mathematical depth; the applied agentic-paradigm track continues regardless.

Session Takeaways

1. The goal of any presentation — and of an elevator pitch — is not to convince in two minutes but to hook the listener into wanting more; depth of insight is the signal everything else decorates.
2. Sub-segment every market: a billion-dollar superset number says nothing about your addressable bucket, and inference ("everyone needs medicines") is not insight.
3. A metric without a denominator is meaningless — express every metric as A by B, and remember that revenue alone, users alone, and "retention" without a time period are all faff.
4. A PRD is judged by the questions it prevents: thorough enough to carry the full essence, succinct enough to be read, never beyond 18-25 pages — and the identical discipline governs LLM prompts.
5. Users read importance from proximity, function from similarity, and meaning from copy; a single word — enroll versus buy — changes how people relate to a product.
6. Cost of change rises with fidelity: contest direction at the sketch and wireframe stage, because after visual design every change reworks four layers at once.
7. Ship through a handshake, not a handoff — engineering and design must shape the BRD while it is a draft, and a genuinely shipped major feature takes about a month.
8. A PM's job is to reduce underutilisation: run the next feature's discovery while the current sprint executes, and give designers the lead time everyone else forgets.
9. The AI revolution is one loop at scale — predict, measure error, backpropagate, adjust — unleashed only when cheap data (Jio) and parallel compute (CUDA) ended the AI winter.
10. LLMs output only text; every image or video is a second model taking dictation — and their most remarkable property is emergent behaviour, doing things nobody trained them to do.
11. Tokens are money: prompt in common, well-trained vocabulary ("it's an accordion"), size prompts optimally, and specify only what is genuinely non-obvious.
12. LLMs are built to answer, not to question — they will confidently hallucinate and certify their own broken work, so verification gates and human proof are non-negotiable.
13. Reasoning is a purchasable number of define-do-verify loops; choose the level deliberately, because more loops buy reliability at real cost.
14. Fundamentals compound and recipes expire: frontier AI knowledge turns over every ten days, so learn the principles once and read Twitter, Substack and arXiv forever.